

PRACTICAL – 2

Implementation and Time analysis of linear and binary search algorithm

Aim:

To implement and analyze the time complexity of Linear Search and Binary Search.

Objective:

The objective of this practical is to implement and compare the performance of **Linear Search** and **Binary Search**. We will analyze the time complexity and execution time of each algorithm to understand their efficiency and identify the most suitable algorithm for different applications.

1.Linear Search

Algorithm:

1. Start.
2. Set $i = 0$.
3. Compare $A[i]$ with the key.
4. If $A[i] = \text{key}$, return the position i .
5. Otherwise, increment i by 1.
6. Repeat steps 3–5 until $i < n$.
7. If the key is not found, return -1.
8. Stop.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n, key;

    cout << "Enter the number of elements: ";
    cin >> n;

    int A[n];

    cout << "Enter the elements of the array: ";
```

```
for (int i = 0; i < n; i++) {
cin >> A[i];
}

cout << "Enter the element to search: ";
cin >> key;

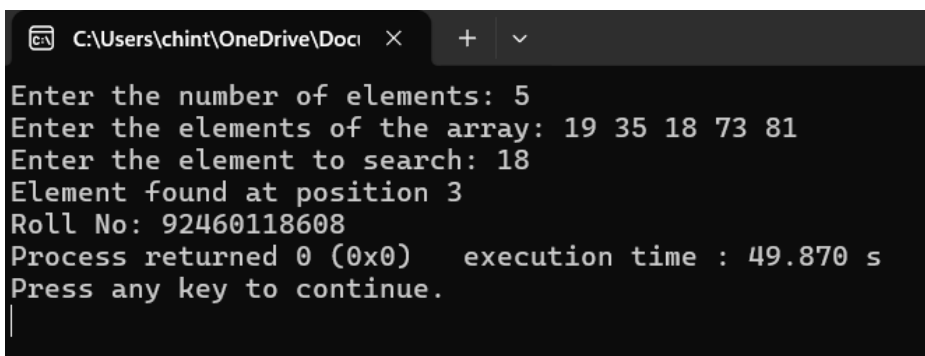
int position = -1;

for (int i = 0; i < n; i++) {
if (A[i] == key) {
position = i;      break;
}
}

if (position != -1) {
cout << "Element found at position " << position + 1;
} else {
cout << "Element not found";
}
// Enter your roll number
cout << endl << "Roll No: 92460118608";

return 0; }
```

Output:



```
C:\Users\chint\OneDrive\Docu >
Enter the number of elements: 5
Enter the elements of the array: 19 35 18 73 81
Enter the element to search: 18
Element found at position 3
Roll No: 92460118608
Process returned 0 (0x0)   execution time : 49.870 s
Press any key to continue.
```

Time Complexity:

Case	Time Complexity
Best Case	$O(1)$
Average Case	$O(n)$
Worst Case	$O(n)$
Space Complexity	$O(1)$

Roll Number: 92460118608

2.Binary Search

Algorithm:

1. Start.
2. Set low = 0 and high = n - 1.
3. Repeat while low <= high.
4. Calculate mid = (low + high) / 2.
5. If A[mid] = key, return the position mid.
6. If key < A[mid], set high = mid - 1.
7. Otherwise, set low = mid + 1.
8. If low > high, the key is not found.
9. Return -1.
10. Stop.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n, key;

    cout << "Enter the number of elements: ";
    cin >> n;

    int A[n];

    cout << "Enter the elements of the array in sorted order: ";
    for (int i = 0; i < n; i++) {
        cin >> A[i];
    }

    cout << "Enter the element to search: ";
    cin >> key;

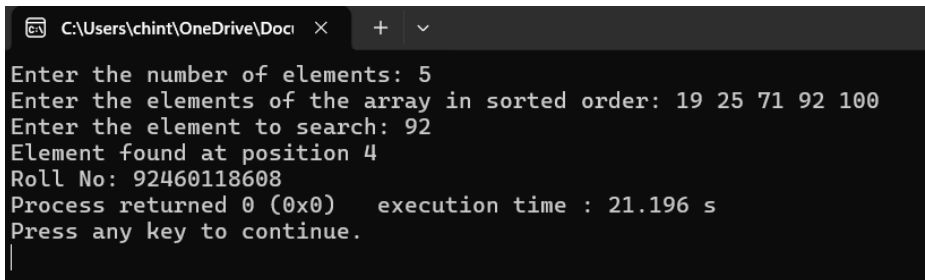
    int low = 0;
    int high = n - 1;
    int position = -1;

    while (low <= high) {
        int mid = (low + high) / 2;
```

```
        if (A[mid] == key) {
position = mid;          break;
        }
        else if (key < A[mid]) {
            high = mid - 1;
        }
    else {
        low = mid + 1;
    }
}

if (position != -1) {
    cout << "Element found at position " << position + 1;
}
else {
    cout << "Element not found";
}
// Enter your roll number
cout << endl << "Roll No: 92460118608";

return 0;
} Output:
```



```
C:\Users\chint\OneDrive\Docu x + v
Enter the number of elements: 5
Enter the elements of the array in sorted order: 19 25 71 92 100
Enter the element to search: 92
Element found at position 4
Roll No: 92460118608
Process returned 0 (0x0)   execution time : 21.196 s
Press any key to continue.
```

Time Complexity:

Case	Time Complexity
Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$
Space Complexity	$O(1)$